

Session 5: Subagents, Hooks, and Final Project

Spawning focused agents, event-driven automation, and an end-to-end research artifact

Brady Allardice

UPF & IBEI

A focused agent spawned for a bounded job —
the right tool for the parallelizable, scoped work
that already shows up in your pipeline.

Concrete patterns, then named teams.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary to your main conversation — not its raw trace

What that buys you:

- ➊ **Parallelism** — run three searches at once instead of one after the other
- ➋ **Context hygiene** — a big messy exploration stays out of your main context budget

Think of it as

Delegating a bounded sub-task to a focused colleague who returns a clean, structured answer — so your main conversation stays on the high-level work.

Where Subagents Shine

Reach for a subagent when the subtask is:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without reading the full trace

Concrete research use cases:

- **Multi-database literature searches** — one agent per database, all running in parallel
- **Coding many documents** against a fixed rubric
- **Parallel robustness checks** across independent specifications
- **Running the same sanity-check** script over many datasets
- **Exploring a codebase** — send a subagent to map out the files, keep the summary in your main context

How to Trigger a Subagent

You don't have to set anything up — just ask.

Claude Code ships with a general-purpose subagent built in. To spawn one, describe the work in plain English:

- *“Send a subagent to map the directory structure”*
- *“Use a subagent to find every place we set the random seed”*

For parallelism, say so explicitly.

- *“Spawn three agents **in parallel** to search Zotero, Google Scholar, and Connected Papers”*
- “In parallel” / “at the same time” / “concurrently” all signal that the agents should run in one batch rather than sequentially

No setup required

The general-purpose agent ships with Claude Code. Defining your own named agent (later this session) is for when you want to reuse a scoped role across many sessions.

Using a Generic Subagent in Practice

Worked example: a parallel literature scan across three sources.

Your prompt to Claude:

Spawn three subagents in parallel:

1. Search my Zotero library for papers using regression discontinuity in education.
2. WebSearch for recent (2023+) arxiv papers on the same topic.
3. WebFetch a few key journal pages I'll list.

Return one combined table: title, year, source, one-sentence claim, link.

What Claude does:

- Spawns three general-purpose subagents in a single message — they run concurrently
- Each searches in its own context window with its own tool calls
- You get one clean table back; the search noise stayed inside the subagents

Three habits that make subagents pay off:

- ❶ **Brief them like a contractor.** Concrete deliverable, all the context they need up front — they don't see your conversation history.
- ❷ **Ask for structured output** (bullet list, table, JSON). You'll be consuming the result; make it easy to read and verify.
- ❸ **Verify independently.** You get a summary, not the full trace. Spot-check the way you would a student's work.

The payoff

A literature sweep, robustness battery, or repo survey that would have taken an hour finishes in minutes — and doesn't eat into your main conversation's context. That's what makes them worth reaching for.

What a Predefined Subagent Looks Like

A small markdown file — author it once, Claude spawns it many times.

Where it lives: `~/.claude/agents/<name>.md` (user-level) or
`.claude/agents/<name>.md` (project-level).

What you specify (YAML frontmatter + body):

- **Identity** — name and a one-line description. The description is what Claude reads to decide when to spawn this agent.
- **Tool allowlist** — exactly which tools it can use (e.g. Read only, no Bash).
- **Model** — Opus, Sonnet, or Haiku.
- **System prompt** — the body of the file: role, disposition, expected output format.

YAML is just a plain-text key: value format — the metadata block sits between two `---` lines at the top of the file.

What YAML Looks Like

A complete predefined subagent fits in eight lines:

```
---  
name: lit-search  
description: Use for multi-database literature searches.  
tools: [mcp__zotero__*, WebSearch, WebFetch]  
model: sonnet  
---  
You are a literature search specialist. Return a structured  
bibliography with the key claim from each paper.
```

Reading it:

- Lines between --- are YAML: one key: value per line; lists in square brackets.
- name, description, tools, model are the four fields you'll use.
- Everything below the second --- is the system prompt — plain English.

Why Define a Named Subagent at All?

If skills are reusable, why not just spawn ad-hoc subagents that pick up the right skill?

The framing

Skills are knowledge. Subagents are roles.

A role bundles a disposition, a tool scope, and an invocation contract — and then draws on skills to execute.

Three things a role gives you that a skill can't:

- ❶ **Tool scope — enforced.** A code-review subagent has read-only access, no bash. Skills are advisory; tool scopes aren't.
- ❷ **Identity, not advice.** The system prompt frames the whole disposition — “find problems, don't fix them.” Subagents *are* the disposition.
- ❸ **A stable invocation contract.** Delegate code review → get a structured critique. Ad-hoc means re-specifying the contract each time.

Pre-define a subagent when the **role** is the reusable unit, not just the know-how.

Building a Specialized Agent

Worked example: turn that lit-search prompt into a reusable role.

Walking through the four fields:

- ❶ **Name + description.** What it does, plus the one-line trigger Claude reads to decide when to spawn it.
“Use for multi-database literature searches on a topic.”
- ❷ **Tool allowlist.** Only what it needs — `mcp__zotero__*`, `WebSearch`, `WebFetch`. No `Bash`, no `Edit`: a search agent shouldn't be writing files.
- ❸ **Model.** `sonnet` — `search-and-synthesize` is moderate weight, no need for `Opus`.
- ❹ **System prompt.** The disposition + the output contract: *“You are a literature search specialist. Return a structured bibliography with the key claim from each paper.”*

Save as `~/.claude/agents/lit-search.md` — you saw the resulting YAML two slides back.

Now in any session:

- Claude auto-spawns it when a task matches the description
- Or invoke it explicitly: *“use the lit-search agent for X”*
- Spawn several in one message — they still run in parallel

Permissions still apply: any tools the agent uses must be allowed in `.claude/settings.json`.